

DATABASE

1. Object Storage

- **Amazon S3** – Can store virtually unlimited images; fast, reliable, pay-per-use.
- **Google Cloud Storage (GCS)** – Similar to S3, integrates well with AI/ML pipelines.
- **Azure Blob Storage** – Enterprise-ready, scalable.
- **MinIO** – Open-source, S3-compatible, good for on-premises deployment.

How it works:

- Store the image itself in object storage.
- Store **metadata** (filename, URL, size, tags) in a database like PostgreSQL or MySQL.

Pros: Scales to billions of images, cheap, fast for uploads/downloads.

Cons: Not a relational DB; you only store references in DB.

2. NoSQL Databases (to store images in DB itself)

- **MongoDB (GridFS)** – Splits images into chunks; handles files >16MB easily.
- **Cassandra** – Highly scalable, good for distributed storage of blobs.

Pros: Horizontal scaling, flexible schema.

Cons: Performance may drop for millions of very large images; DB size grows fast.

3. Traditional SQL Databases (Not ideal, only if necessary)

- **PostgreSQL** – BYTEA or Large Objects (LOB), can handle GB-scale images.
- **Oracle** – BLOB with SecureFiles.
- **MySQL** – LONGBLOB.

Limitation: Managing millions of large images **inside a relational DB is inefficient**. Backup/restore becomes slow, DB size balloons, and performance drops.

Summary:

- **For millions of images → Object Storage + relational DB for metadata.**

Popular APIs that store images directly

1. Cloudinary (VERY popular)

- Stores images & videos
- Auto compression & resizing
- Returns image URL

2. AWS S3 (Object Storage API)

- Stores images as objects
- Extremely scalable
- Used by enterprises

upload via API → get an S3 URL

→ Industry standard

3. Google Cloud Storage

- Similar to AWS S3
- REST API based
- Secure & fast

4. Azure Blob Storage

- Microsoft cloud storage
- REST API available

5. Firebase Storage

- Built on Google Cloud
- Simple API for apps
- Good for Android/Web apps

6. Supabase Storage

- Open-source friendly
- Easy API
- Returns public/private URLs

- ❖ *MySQL was chosen for reliable structured data management, and MinIO was chosen for efficient and scalable file storage. Using both ensures better performance, lower cost, and clean system design.*

Why MinIO is used:

- **Faster and safer** than storing files in a database
- **Scalable** — can store a large number of files
- **S3-compatible** — same style as AWS S3
- **Can run locally using Docker** (no cloud cost)

- ❖ *MinIO is a high-performance, S3-compatible object storage system used to store and manage large files such as images and media efficiently.*

Why MySQL was used (and not other databases)

Pros (Why MySQL)

- **Best fit for structured data**
User details, records, and relationships are clearly structured → MySQL fits naturally.
- **Reliable and consistent**
ACID support ensures correct and safe data updates.
- **Simple and lightweight**
Easy to set up, manage, and maintain for a medium-scale website.
- **Industry standard**
Widely used, stable, and well-supported.

Cons (Why not others)

- **Not ideal for unstructured data**
Less flexible than NoSQL for schema-less data.
- **Limited for very large-scale systems**
Harder to scale compared to some NoSQL solutions.

Why not others:

- **MongoDB** → weaker for relational data
- **PostgreSQL** → more complex than required
- **Oracle / SQL Server** → costly and heavy
- **SQLite** → not suitable for multiple users

Why MinIO was used (and not other storage options)

Pros (Why MinIO)

- **Designed for file storage**
Best for images, media, and large files.
- **High performance**
Faster uploads/downloads than databases.
- **Self-hosted and cost-free**
No cloud cost during development or POC.
- **S3-compatible**
Easy future migration to cloud storage.

Cons (Why not others)

- **Cannot manage structured data**
Needs a database like MySQL for metadata.
- **Extra service to manage**
Requires separate setup and configuration. (MinIO needs separate setup like running it in Docker and creating buckets for storing images.)

Why not others:

- **Storing files in MySQL** → slows database
- **Local file system** → not scalable or safe
- **AWS S3** → unnecessary cost at early stage
- **FTP servers** → poor scalability & security

Why MySQL + MinIO together

- **MySQL** → stores structured data & file metadata
- **MinIO** → stores actual image/media files
- Prevents database overload
- Improves performance and scalability
- Follows real-world industry architecture